

JENKINS TASK-2

What is Jenkins?

Jenkins is an open-source automation server used to build, test, and deploy applications automatically.

It is widely used in CI/CD pipelines to help developers deliver software faster, reliably, and with fewer errors.

Advantages of Jenkins

1. Automation – Automates build, test, and deployment processes, reducing manual work.
 2. CI/CD Support – Enables Continuous Integration and Continuous Delivery for faster releases.
 3. Extensible – Thousands of plugins available for Git, Docker, AWS, Maven, Kubernetes, etc.
 4. Open Source & Free – No licensing cost and strong community support.
 5. Easy Integration – Integrates easily with version control tools, cloud platforms, and DevOps tools.
 6. Scalable – Supports distributed builds using master-agent architecture.
 7. Improves Code Quality – Detects issues early through automated testing.
-

One-line interview answer

Jenkins is an open-source CI/CD tool that automates software build, testing, and deployment to enable faster and reliable application delivery.

1. Configure 2 slave machines in Jenkins master.

Task 1: Configured 2 Slave Machines in Jenkins Master

- Installed Java on two separate Linux servers to prepare them as Jenkins build agents.
- From Jenkins master, navigated to **Manage Jenkins → Nodes** and created two **Permanent Agents**.
- Configured each node with a **remote root directory** (create one directory) and network connectivity details.

- Used SSH keys to securely authenticate and connect to the agent machines.
- Assigned appropriate **labels** to each agent for targeted job execution.
- Verified connectivity and ensured both agents showed a **stable “Online”** status in Jenkins UI.
- Successfully enabled distributed builds, improving scalability and workload balancing.

✓ **Created slave 1 and slave 2 machines and installed java in both servers**

```

[ec2-user@ip-172-31-23-190 ~]$ sudo yum install -y java-17-amazon-corretto
java-version
Amazon Linux 2023 Kernel Livepatch repository
Dependencies resolved.
257 kB/s | 29 MB   00:00

Package                               Architecture      Version            Repository         Size
----                               -
Installing:
java-17-amazon-corretto               x86_64            1:17.0.17+0-1.amzn2023.1  amazonlinux        219 k
Installing dependencies:
alsa-lib                              x86_64            1:2.7.2-1.amzn2023.0.2  amazonlinux        504 k
cairo                                  x86_64            1:18.0-4.amzn2023.0.3  amazonlinux        719 k
dejavu-mono-fonts                     noarch           2.37.16.amzn2023.0.2  amazonlinux        1.3 M
dejavu-mono-fonts                     noarch           2.37.16.amzn2023.0.2  amazonlinux        467 k
dejavu-serif-fonts                    noarch           2.37.16.amzn2023.0.2  amazonlinux        1.9 M

Transaction Summary
Install 35 Packages
Total download size: 100 M
Installed size: 262 M
Downloading Packages:
(1/35): alsa-lib-1:2.7.2-1.amzn2023.0.2.x86_64.rpm                9.4 MB/s | 504 kB   00:00
(2/35): dejavu-mono-fonts-2.37.16.amzn2023.0.2.noarch.rpm       20 MB/s | 1.3 MB   00:00
(3/35): cairo-1:18.0-4.amzn2023.0.3.x86_64.rpm                 8.8 MB/s | 719 kB   00:00
(4/35): dejavu-mono-fonts-2.37.16.amzn2023.0.2.noarch.rpm       14 MB/s | 467 kB   00:00
(5/35): dejavu-serif-fonts-2.37.16.amzn2023.0.2.noarch.rpm      25 MB/s | 1.0 MB   00:00
(6/35): fontconfig-2.13.9-2.amzn2023.0.2.x86_64.rpm            389 MB/s | 9.2 kB   00:00
(7/35): fontconfig-2.13.9-2.amzn2023.0.2.x86_64.rpm             7.7 MB/s | 273 kB   00:00
(8/35): freetype-2.13.2-5.amzn2023.0.1.x86_64.rpm              1.5 MB/s | 48 kB    00:00
(9/35): google-noto-fonts-common-20240401.1.amzn2023.0.2.noarch.rpm 10 MB/s | 423 kB   00:00
(10/35): google-noto-mono-xf-fonts-20240401.1.amzn2023.0.2.noarch.rpm 452 MB/s | 57 kB    00:00
(11/35): google-noto-mono-xf-fonts-20240401.1.amzn2023.0.2.noarch.rpm 22 MB/s | 593 kB   00:00
(12/35): gresplit2-1.3.14-7.amzn2023.0.2.x86_64.rpm             3.3 MB/s | 97 kB    00:00
(13/35): harfbuzz-7.0.0-2.amzn2023.0.2.x86_64.rpm              13 MB/s | 873 kB   00:00
(14/35): java-17-amazon-corretto-17.0.17+0-1.amzn2023.1.x86_64.rpm 7.7 MB/s | 219 kB   00:00
(15/35): java-runtime-filesystems-4.0.0-7.amzn2023.0.4.noarch.rpm 369 MB/s | 12 kB    00:00
(16/35): langpacks-core-font-en-3.0.21.amzn2023.0.4.noarch.rpm 293 MB/s | 10 kB    00:00
(17/35): libICE-1.1.1-3.amzn2023.0.1.x86_64.rpm                 2.8 MB/s | 74 kB    00:00
(18/35): libSM-1.2.4-2.amzn2023.0.1.x86_64.rpm                  1.1 MB/s | 45 kB    00:00

```

- ✓ **Created two agents one for java one for python label**
- ✓ **Add remote directory, create on directory in slave-1**
- ✓ **Launch via ssh method and Host as public ID of the slave-1**
- ✓ **Add credentials then select ssh method give user name, private key as ssh key**
- ✓ **Passphrase as empty and Id/Description as agent-ssh-key**

Jenkins / Manage Jenkins / Nodes / New node

New node

Node name

Slave_1

Type

☒ Permanent Agent

Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create

Jenkins Credentials Provider: Jenkins

SSH key for Jenkins agent2

Username
jenkins2

☐ Treat username as secret ?

Private Key
☒ Enter directly

Key

Enter New Secret Below

```
BR19c7BAHgq2c+UrDyO+VlHpmh8w/jwkpharUTLFj*IR9kNoAAADBAQJ56BFPgpFFw+gI
RrGdQMR71s5PQU8Q7Hfz+sQduOYXX0vzZQMLPHZc1J+oQgT80ko/E13k4Mf183-1tFSrY
p06C0myf36e/YIm/DE131C1phuMucoHauG9qQAmDatuxHnZ2/vV1G9DcCWL/ElnSIUGko1
oXVKaLUdJH920AGGJHfWg0182pDhFfcRfOBVwJ513juZr6H4y5SQDS50R0z23Hjz2p+HKGa
EaFo1b+wrJTA1rH8msxxAx2/wZCC4SHQAAMEAy2Y5gPnoW5Y0t1KJysFBBGwOPxEJL2ZJ
GZVeaK6Ee9QdLpTYmhg5e1TBRcAg3/3BuAbytdVyzvuCNZ/3eKDQkr115A1TYyJb88NU
1538s5Q05UPHatrH4s7u76C+SmQJBBH4XskYtDYG1m15BzYHf5Z8n667/1CvKdweK1wC
3UmT142i1foa9FuCrU200+2Kjq3NRTFuimq1wvLDK8VLP65m25ZFCsKpyWwv5eOr1NTyCy
0wao3HvWfBT1EVA5AA3Wp1bmtpbmNaAXAERTcyLTpkLTizLEY0551YzIuaH50ZK3uYwWb
AgMEBQ==
-----END OPENSSH PRIVATE KEY-----
```

Ssh key generated and saved id_rsa.pub key in authorized_keys file using command and given permissions

```
Last login: Wed Dec 24 09:50:35 2025 from 18.206.107.27
[ec2-user@ip-172-31-23-190 ~]$ sudo su -
[root@ip-172-31-23-190 ~]# java --version
openjdk 17.0.17 2025-10-21 LTS
OpenJDK Runtime Environment: Corretto-17.0.17.10.1 (build 17.0.17+10-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.17.10.1 (build 17.0.17+10-LTS, mixed mode, sharing)
[ec2-user@ip-172-31-23-190 ~]# ssh-keygen
ssh: Could not resolve hostname keygen: Name or service not known
[ec2-user@ip-172-31-23-190 ~]# ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:rpQ01ab85Cq+V2bK3KagpKph8Kf17am20hQUMuo root@ip-172-31-23-190.ec2.internal
The key's randomart image is:
+--[RSA 3072]-----+
| ooo+ |
| o.+. |
| o.+. |
| o.+. |
| o.+. |
| o.+. |
| o.+. |
| o.+. |
| o.+. |
| o.+. |
+-----+
[ec2-user@ip-172-31-23-190 ~]# cd .ssh
[ec2-user@ip-172-31-23-190 .ssh]# ls
authorized_keys  id_rsa  id_rsa.pub
[ec2-user@ip-172-31-23-190 .ssh]# cat id_rsa.pub > authorized_keys
[ec2-user@ip-172-31-23-190 .ssh]# chmod 700 authorized_keys
[ec2-user@ip-172-31-23-190 .ssh]#
```

i-0fe5d64593c107761 (Slave-1)
PublicIPs: 18.208.212.217 PrivateIPs: 172.31.23.190

Done same with slave_2 Ssh key generated and saved id_rsa.pub key in authorized_keys file using command and given permissions

```
/m/
Last login: Wed Dec 24 09:50:42 2025 from 18.206.107.28
[ec2-user@ip-172-31-23-129 ~]$ ssh-key
-bash: ssh-key: command not found
[ec2-user@ip-172-31-23-129 ~]$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/ec2-user/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/ec2-user/.ssh/id_rsa
Your public key has been saved in /home/ec2-user/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:ANW6uaidUQcGiatAcem+rWgboasLqK2RM8kWpXsvc ec2-user@ip-172-31-23-129.ec2.internal
The key's randomart image is:
+--[RSA 3072]-----+
| .+ |
| ..B. |
| o.o.o |
| +oo+.o |
| +..+o+.S |
| ..+o |
| B++o. |
| B++o. |
| B++o. |
| B++o. |
+-----+
[ec2-user@ip-172-31-23-129 ~]$ cd .ssh
[ec2-user@ip-172-31-23-129 .ssh]# ls
authorized_keys  id_rsa  id_rsa.pub
[ec2-user@ip-172-31-23-129 .ssh]# cat id_rsa.pub > authorized_keys
[ec2-user@ip-172-31-23-129 .ssh]# chmod 700 authorized_keys
[ec2-user@ip-172-31-23-129 .ssh]#
```

i-006ffa140c2965c62 (Slave-2)
PublicIPs: 34.227.46.106 PrivateIPs: 172.31.23.129

In jenkins master machine created a directory called .ssh, mkdir /var/lib/Jenkins/.ssh

```

Last login: Wed Dec 24 09:11:09 2025 from 27.116.18.99
[ec2-user@ip-172-31-21-117 ~]$ sudo su -
Last login: Wed Dec 24 09:23:49 UTC 2025 on pts/1
[root@ip-172-31-21-117 ~]# java --version
Unrecognized option: --version
Error: Could not create the Java Virtual Machine.
Error: A fatal exception has occurred. Program will exit.
[root@ip-172-31-21-117 ~]# sudo yum install -y java-17-amazon-corretto
java -version
Last metadata expiration check: 20:45:34 ago on Tue Dec 23 13:33:31 2025.
Package java-17-amazon-corretto-1:17.0.17-10.1.amzn2023.1.x86_64 is already installed.
Dependencies resolved.
Nothing to do.
Complete!
openjdk version "17.0.17" 2025-10-21 LTS
OpenJDK Runtime Environment Corretto-17.0.17-10.1 (build 17.0.17+10-LTS)
OpenJDK 64-Bit Server VM Corretto-17.0.17-10.1 (build 17.0.17+10-LTS, mixed mode, sharing)
[root@ip-172-31-21-117 ~]# cd /var/lib/
[root@ip-172-31-21-117 lib]# cd jenkins/
[root@ip-172-31-21-117 jenkins]# mkdir .ssh
[root@ip-172-31-21-117 jenkins]#

```

SSH-keyscan for both slave machines and saved it in known_hosts and given permissions and changed owner as well

```

[root@ip-172-31-21-117 .ssh]# ssh-keyscan -H 18.208.212.217 >> /var/lib/jenkins/.ssh/known_host
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
[root@ip-172-31-21-117 .ssh]# ssh-keyscan -H 18.208.212.217 >> /var/lib/jenkins/.ssh/known_hosts
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
[root@ip-172-31-21-117 .ssh]# ls
known_host  known_hosts
[root@ip-172-31-21-117 .ssh]# rm known_host
rm: remove regular file 'known_host'?
[root@ip-172-31-21-117 .ssh]# ls
known_host  known_hosts
[root@ip-172-31-21-117 .ssh]# rm -rf known_host
[root@ip-172-31-21-117 .ssh]# ls
known_hosts
[root@ip-172-31-21-117 .ssh]# ssh-keyscan -H 18.208.212.217 >> /var/lib/jenkins/.ssh/known_hosts
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
# 18.208.212.217:22 SSH-2.0-OpenSSH_8.7
[root@ip-172-31-21-117 .ssh]# chown -R jenkins:jenkins /var/lib/jenkins/.ssh
[root@ip-172-31-21-117 .ssh]# chmod 700 /var/lib/jenkins/.ssh
[root@ip-172-31-21-117 .ssh]# chmod 600 /var/lib/jenkins/.ssh/known_hosts
[root@ip-172-31-21-117 .ssh]#

```

S	Name ↓	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Agent-1	Linux (amd64)	In sync	17.95 GiB	0 B	17.95 GiB	43ms

AGENT-2 / Slave -2 server:

- ✓ Created two agents one for java one for python label
- ✓ Add remote directory, create on directory in slave-2
- ✓ Launch via ssh method and Host as public ID of the slave-2
- ✓ Add credentials then select ssh method give user name, private key as ssh key
- ✓ Passphrase as empty and Id/Description as agent-ssh-key

Name ?

slave-2

Description ?

Plain text [Preview](#)

Number of executors ?

1

Remote root directory ?

/home/jenkins/slave-workspace

Labels ?

python

Jenkins Credentials Provider: Jenkins

SSH key for Jenkins agent2

Username

jenkins2

☐ Treat username as secret ?

Private Key

☒ Enter directly

Key

Enter New Secret Below

```
BR19c7BAHgqZc+UrDyO+VtHpmh8w/jwkphrUTLF3pTR9kNoAAADBAQJ56BFPpFFW+gI
Rr+dgRE71sPQUBQ7HfzExQuoDYXXoveZQPLfH2x1J+oQgT8Bco/E1JkUwF183+1tESCrY
p06C0myf36e/YIm/DE131C1pHuuoOhaugQAmDatwHnZ2/vV1G9DeCWL/E1n5IUGko1
pXVKqLudJh9Q6xGkRfWg0182pDpFfcRf0RYwJ513juzn6NpSSQDSOR0JZ3Nj3zP+NRGa
Eaf03bwerJTA1nHBmss+shgZvZC45RGAAMeAy2V8pFmomyV01K3ysPFBmouQPE1L2J1
GZVeGaK6E9QdLpTYmhG5e1TBRCgAg3/3BuabyDVvzvUOWZ/3eKDQkr115A1TYyJ688NU
LS5S5G05UPBat+R4+7u76c+5eQ3BBh4XkyIDGtmJ55BzYNSzBm667/3CvK6eK1vK
3UeT34213Foa9FucrU200+2K3qjNREfuiq1wLdkRVLp6Sm25ZFCsKpy0wvSeDr3NTyCy
08eo3Hyf0E1EVAAAQ9a3lmtgbenNaGARTcyLTh6LT1d1TEY0531YzEuaw0ZChYw6B
AgMEBQ==
-----END OPENSSH PRIVATE KEY-----
```

slave-2	N/A	N/A	N/A	N/A	N/A
Data obtained	13 min	13 min	13 min	13 min	13 min

Icon: S M L Legend

2. Configure webhooks to Jenkins job.

- Enabled **GitHub hook trigger for GITScm polling** inside Jenkins job configuration.
- In GitHub repository, opened **Settings** → **Webhooks** and added a Jenkins webhook.
- Used Jenkins public URL formatted as:
http://<public-ip>:8080/github-webhook/
- Set content type to **application/json** for proper payload handling.
- Selected the **push event** to trigger build whenever code gets pushed.
- Tested by pushing a commit and observed Jenkins job triggering automatically.
- Achieved seamless CI automation between GitHub and Jenkins.

✓ Created new job for webhook

New Item

Enter an item name

webhook_job

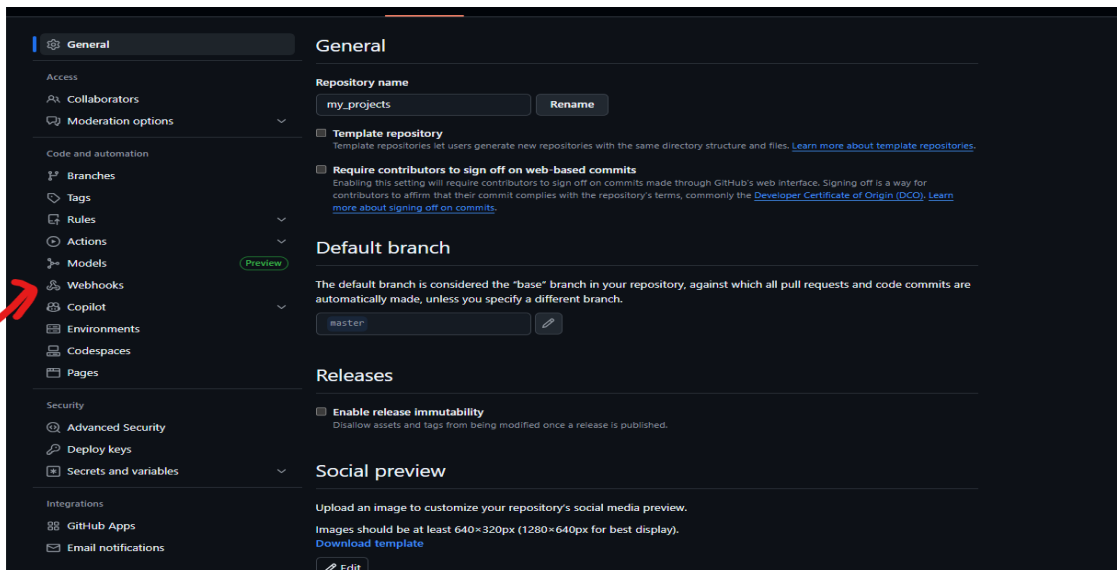
Select an item type



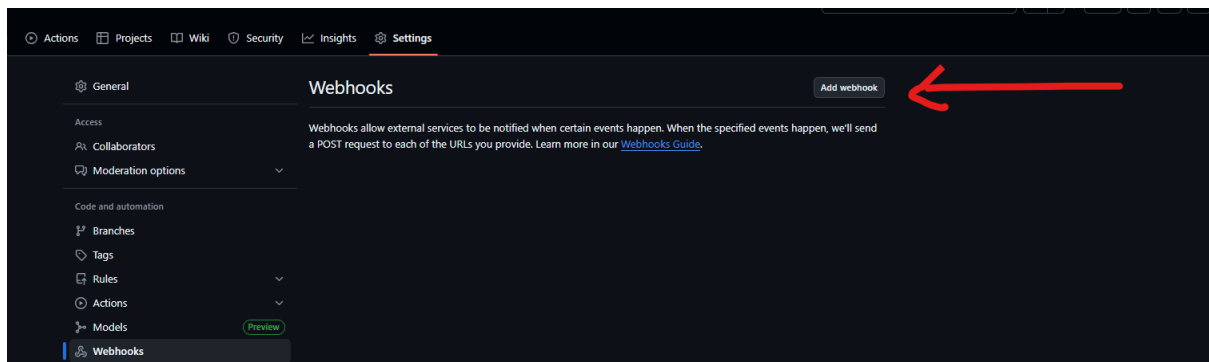
Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

- ✓ Go to GITHUB Repo and select webhook and http://<public-ip>:8080/github-webhook/



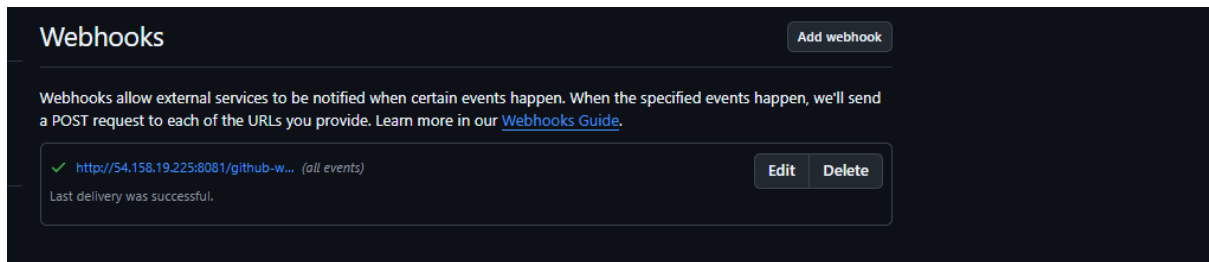
✓ Selected webhook



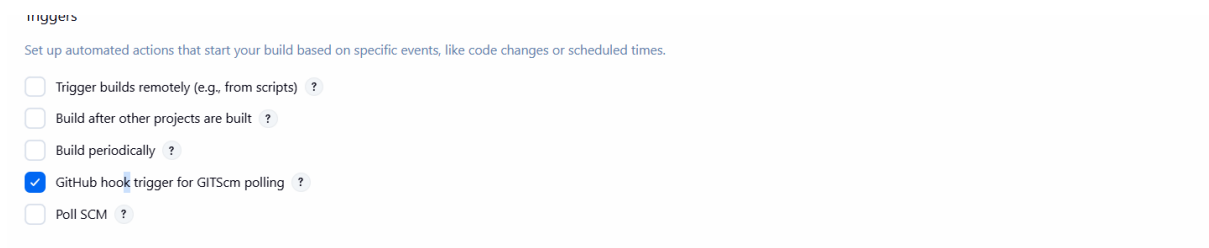
✓ Add `http://<public-ip>:8080/github-webhook/`

This is a screenshot of the 'Webhooks / Add webhook' form in GitHub. The form contains several sections: 'Payload URL' with the value 'http://54.158.19.225:8081/github-webhook/', 'Content type' set to 'application/x-www-form-urlencoded', an empty 'Secret' field, 'SSL verification' with 'Enable SSL verification' selected, and a section for 'Which events would you like to trigger this webhook?' with 'Send me everything.' selected. At the bottom, there is an 'Active' checkbox which is checked, and a green 'Add webhook' button. A red arrow points to this button.

✓ Webhook is created



Created a job by selecting github hook trigger



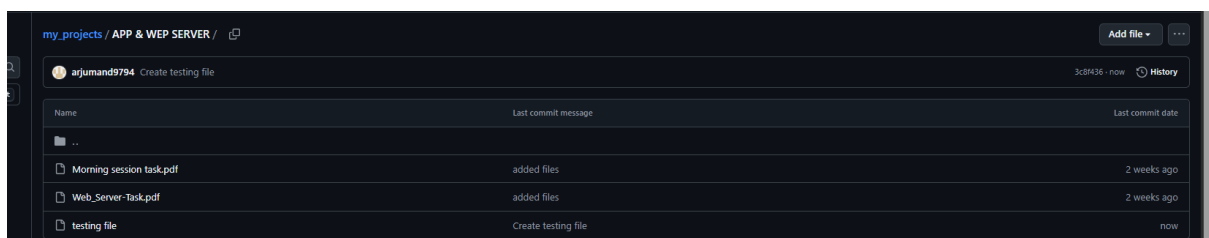
Environment

Created the job successfully adding URL of the git repo

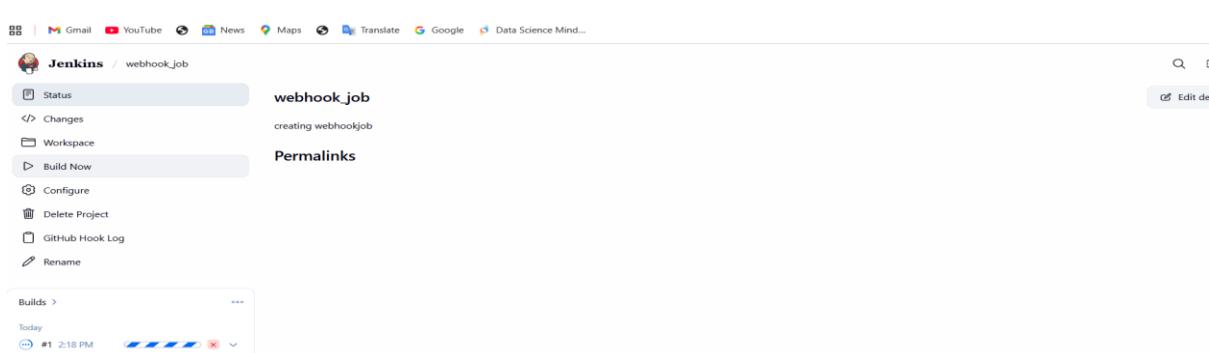


Validation:

Creating a file for testing purpose



Job triggers without human intervention



✓ Validated job is executed



Started by user [Arjumand Mohiuddin](#) Started 10 sec ago
Took 4.5 sec on Agent-1

This run spent:

- 6 ms waiting;
- 4.5 sec build duration;
- 4.5 sec total from scheduled to completion.

git **Revision:** 289cb385bbaa4f0637ba795535b69dadee7122ea
Repository: https://github.com/arjumand9794/my_projects.git

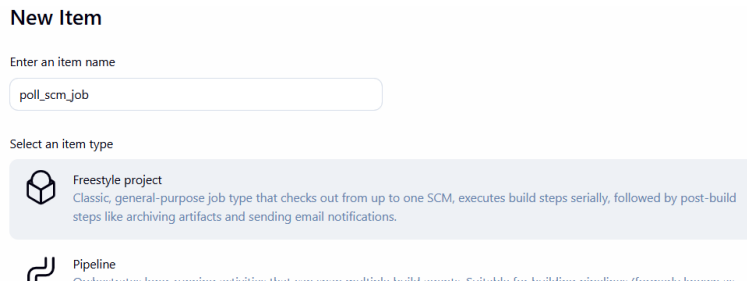
- refs/remotes/origin/master

</> No changes.

3. Configure poll scm and build periodical options in Jenkins job.

- Opened Jenkins job configuration and navigated to **Build Triggers** section.
- Enabled **Poll SCM** and configured schedule using cron syntax:
* * * * * (checks repo every 5 mins for new commits).
- Enabled **Build Periodically** and scheduled:
* * * * * (builds daily at 2 AM).
- Verified polling by pushing changes and monitoring job trigger logs.
- Ensured periodic automation for maintenance and continuous validation builds.
- Improved job reliability with automatic scheduled and change-driven execution.
- This setup helps reduce manual intervention and supports continuous delivery workflows.

✓ Created new job for poll_scm



New Item

Enter an item name

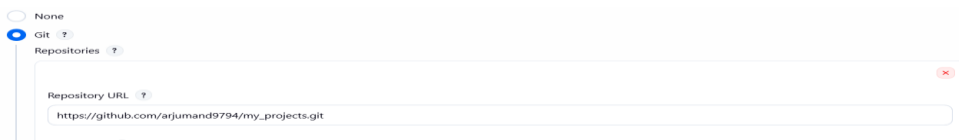
poll_scm_job

Select an item type

☒ Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

☐ Pipeline
A declarative language for defining jobs that can run on Jenkins agents. Suitable for building reusable frameworks.

✓ Selected git repo url and option poll_scm



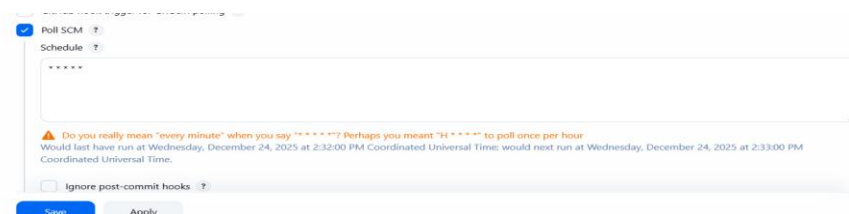
☐ None
☒ Git

Repositories

Repository URL [?](#)

https://github.com/arjumand9794/my_projects.git

✓ Given all 5 stars which mean it will goto git each minute and check for changes



☒ Poll SCM [?](#)

Schedule [?](#)

* * * * *

⚠ Do you really mean "every minute" when you say "* * * * *"? Perhaps you meant "H * * * * *" to poll once per hour
Would last have run at Wednesday, December 24, 2025 at 2:32:00 PM Coordinated Universal Time; would next run at Wednesday, December 24, 2025 at 2:33:00 PM Coordinated Universal Time.

☐ Ignore post-commit hooks [?](#)

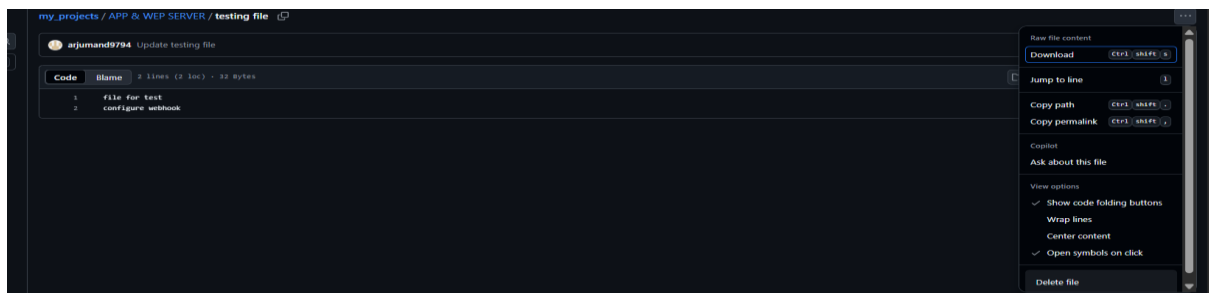
[Save](#) [Apply](#)

Created job lets make some changes in repo so it will start working



Validation:

Deleting the file



Code is executed and validation successful



✓ Created new job for periodical it will execute job based on cronjob we will give

New Item

Enter an item name

periodical_job

Select an item type



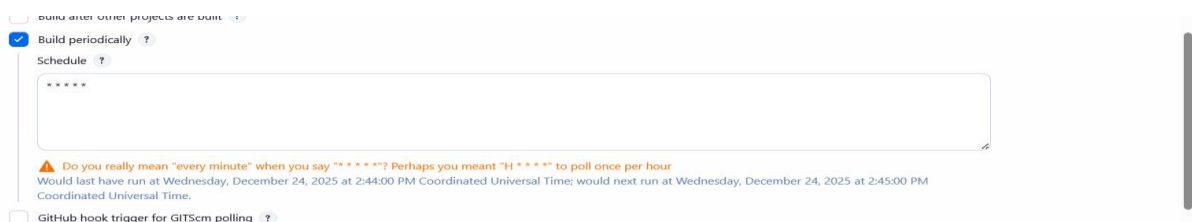
Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

✓ All 5 stars give it will execute job each minute



Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute shell ?

Command

See the list of available environment variables

sleep 15

✓ Validation: Job was executed in periodically

Jenkins

prediodical_job

Status

Changes

Workspace

Build Now

Configure

Delete Project

Rename

prediodical_job

creating prediodical

Permalinks

Builds >

Today

#1 2:49 PM

Edit description

4. Take backup of Jenkins server by using bash script.

- Created a custom backup script under /opt/ to archive Jenkins configuration data.
- Script compressed critical directories including /var/lib/jenkins and service configuration files.
- Enabled execution using:
chmod +x /opt/jenkins_backup.sh.
- Ran the script and verified backup files were generated under /backup/jenkins.
- Ensured backup filenames included dates for easy identification and restoration.
- Automated backup process through cronjobs for scheduled protection.
- This backup serves as a disaster recovery mechanism for Jenkins master.

```
#!/bin/bash

# Variables
JENKINS_HOME="/var/lib/jenkins"
BACKUP_BASE="/backup/jenkins"
DATE=$(date +%Y-%m-%d_%H-%M-%S)
BACKUP_DIR="$BACKUP_BASE/jenkins_backup_$DATE"

# Create backup directory
mkdir -p "$BACKUP_DIR"

# Backup Jenkins home
tar -czf "$BACKUP_DIR/jenkins_home.tar.gz" "$JENKINS_HOME"

# Optional: backup Jenkins config (if exists)
if [ -f /etc/sysconfig/jenkins ]; then
    cp /etc/sysconfig/jenkins "$BACKUP_DIR/"
fi

if [ -f /etc/default/jenkins ]; then
    cp /etc/default/jenkins "$BACKUP_DIR/"
fi

# Cleanup old backups (older than 7 days)
find "$BACKUP_BASE" -type d -mtime +7 -exec rm -rf {} \;

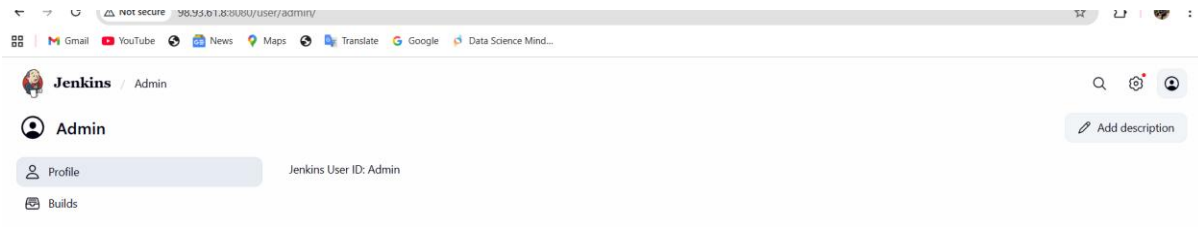
echo "Jenkins backup completed successfully at $BACKUP_DIR"
```

Created backup successfully

```
[ec2-user@ip-172-31-21-117 ~]$ vi jenkins_backup.sh
[ec2-user@ip-172-31-21-117 ~]$ chmod +x jenkins_backup.sh
[ec2-user@ip-172-31-21-117 ~]$ ./jenkins_backup.sh
mkdir: cannot create directory '/backup': Permission denied
tar: Removing leading '/' from member names
tar (child): /backup/jenkins/jenkins_backup_2025-12-24-15-18-07/jenkins_home.tar.gz: Cannot open: No such file or directory
tar (child): Error is not recoverable: exiting now
tar: /var/lib/jenkins/.java/fonts/17.0.17/FontInfo-1-ip-172-31-21-117.ec2.internal-Amazon Linux-2023-en- properties: Cannot open: Permission denied
tar: /backup/jenkins/jenkins_backup_2025-12-24-15-18-07/jenkins_home.tar.gz: Cannot write: Broken pipe
tar: child returned status 2
tar: Error is not recoverable: exiting now
find: /backup/jenkins: no such file or directory
Jenkins backup completed successfully at /backup/jenkins/jenkins_backup_2025-12-24-15-18-07
```

Validation:

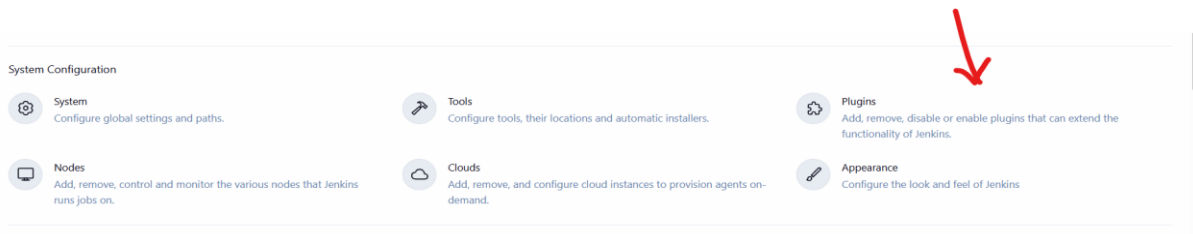
```
tar: Removing leading '/' from member names
jenkins backup completed successfully at /backup/jenkins/jenkins_backup_2025-12-24_15-25-44
[ec2-user@ip-172-31-21-117 ~]$ ls -lh /backup/jenkins
total 0
drwxr-xr-x. 2 ec2-user ec2-user 33 Dec 24 15:24 jenkins_backup_2025-12-24_15-24-32
drwxr-xr-x. 2 root root 33 Dec 24 15:25 jenkins_backup_2025-12-24_15-25-44
[ec2-user@ip-172-31-21-117 ~]$
```



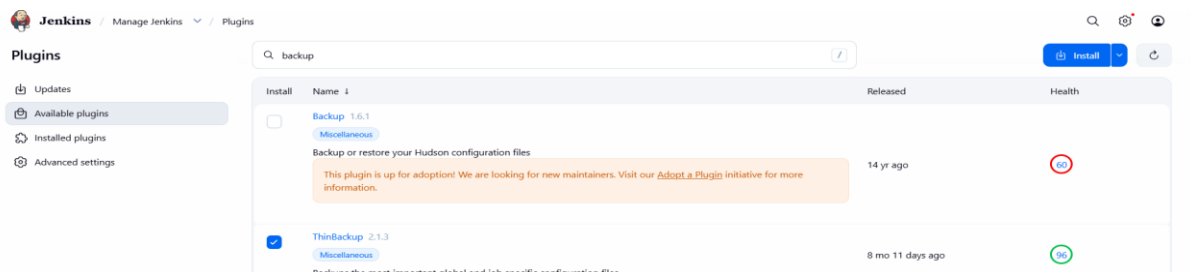
5. Take backup of Jenkins using thin backup plugin.

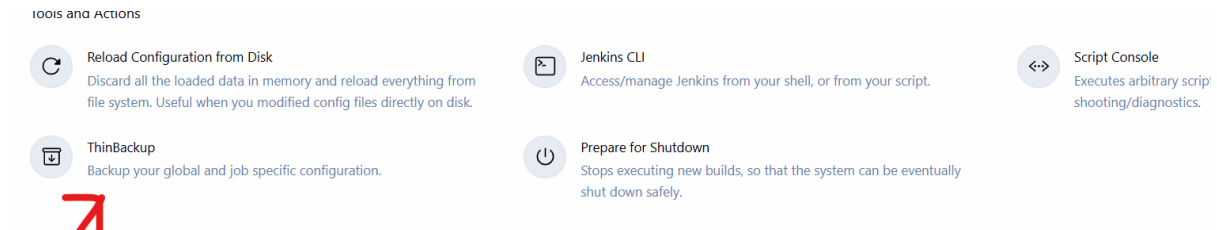
- Installed and configured **ThinBackup plugin** from Manage Plugins.
- Defined a backup storage path such as `/backup/thinbackup/`.
- Performed a **full backup** including jobs, credentials metadata, configurations, and plugins.
- Validated backup result by checking archived files in target backup directory.
- ThinBackup provides backup integrity via incremental and scheduled backup options.
- Plugin-based backup simplifies future maintenance and restores easily on new server.
- Ensured Jenkins can be restored quickly in case of corruption or server failure.

✓ Go to manage and plugin to install thinbackup



✓ Installed back up and then go to thin backup and click for backup





Click on thin backup



6. Setup a new Jenkins server and dump the backup taken in task4.

- Launched a new Jenkins server and installed the required Java and Jenkins services.
- Stopped Jenkins service before restoration to prevent file conflict.
- Copied backup files from first server to new instance using **scp** secure transfer.
- Restored backup contents to `/var/lib/jenkins` and updated permissions for Jenkins user.
- Restarted Jenkins and confirmed that jobs, nodes, and configurations were restored successfully.
- Verified functional access by triggering builds and confirming agent connections.
- This validated that the backup and restore strategy works perfectly for production migration.

Installed new server for backup

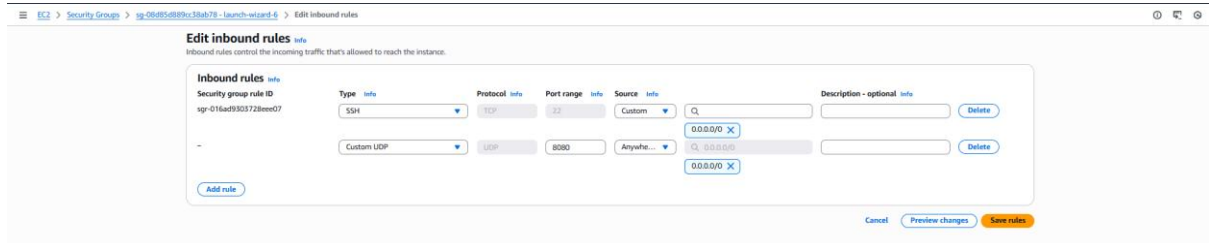
1. [ec2-user ~]\$ `sudo yum update -y`
2. Add the Jenkins repo using the following command:
3. [ec2-user ~]\$ `sudo wget -O /etc/yum.repos.d/jenkins.repo \`
`https://pkg.jenkins.io/redhat-stable/jenkins.repo`
4. Import a key file from Jenkins-CL to enable installation from the package:
[ec2-user ~]\$ `sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key`
[ec2-user ~]\$ `sudo yum upgrade`
5. Install Java:
[ec2-user ~]\$ `sudo yum install java-21-amazon-corretto -y`
6. Install Jenkins:
[ec2-user ~]\$ `sudo yum install jenkins -y`
7. Enable the Jenkins service to start at boot:

```
[ec2-user ~]$ sudo systemctl enable jenkins
```

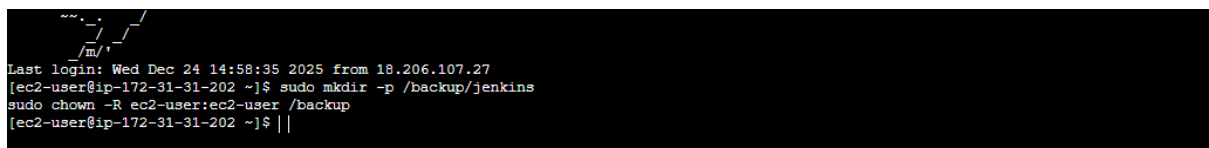
8. Start Jenkins as a service:

```
[ec2-user ~]$ sudo systemctl start Jenkins
```

- ✓ Allowed 8080 portnumber in custom tcp initially I have given custom udp, so it was not working so later I gave custom tc



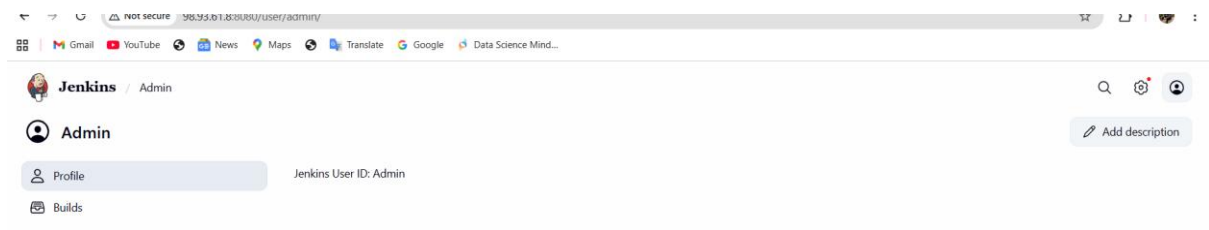
Created backup directory in Destination server



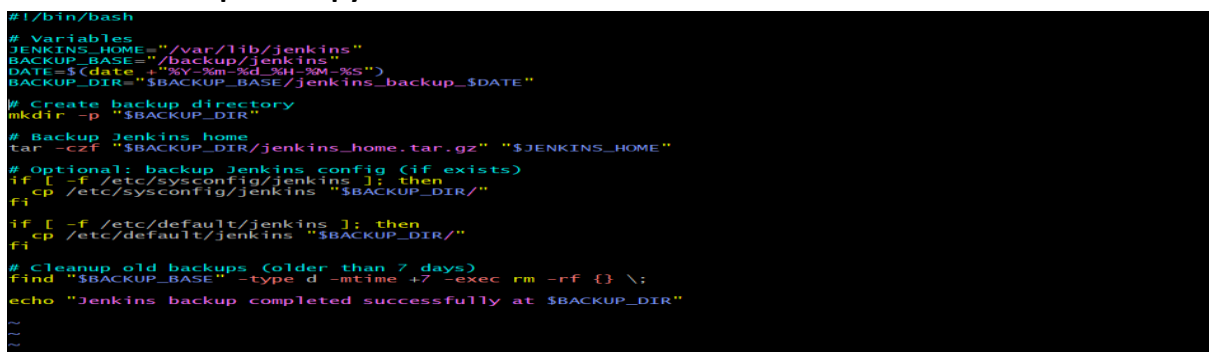
✓ Installed java and jenkins



✓ New Jenkins server is opened after setup credentials



✓ Bash script for copy file from one server and send to another server.



✓ Bashscript executed

```
ec2-user@ip-172-31-21-117 ~]$ vi jenkins_backup.sh
ec2-user@ip-172-31-21-117 ~]$ vi jenkins_backup.sh
ec2-user@ip-172-31-21-117 ~]$ chmod +x jenkins_backup.sh
ec2-user@ip-172-31-21-117 ~]$ sudo ./jenkins_backup.sh
2025: Removing leading '/' from member names
jenkins backup completed successfully at /backup/jenkins/jenkins_backup_2025-12-24_16-25-07
ec2-user@ip-172-31-21-117 ~]$
```

Validation:

✓ Backup received successfully

```
Amazon Linux 2023
https://aws.amazon.com/linux/amazon-linux-2023

Last login: Wed Dec 24 16:58:28 2025 from 10.206.107.27
ec2-user@ip-172-31-21-202 ~]$ sudo mkdir -p /backup/jenkins
ec2-user@ip-172-31-21-202 ~]$ cd /backup
ec2-user@ip-172-31-21-202 ~]$ vi test_xxx
ec2-user@ip-172-31-21-202 ~]$ ls -lh /backup/jenkins
total 0
-rw-r--r-- 1 ec2-user ec2-user 18 Dec 24 16:58 /backup/jenkins/test_xxx
ec2-user@ip-172-31-21-202 ~]$ cd /backup/jenkins
ec2-user@ip-172-31-21-202 jenkins]$ ls
total 0
-rw-r--r-- 1 ec2-user ec2-user 18 Dec 24 16:58 /backup/jenkins/test_xxx
ec2-user@ip-172-31-21-202 jenkins]$ ls -lh /backup/jenkins
total 0
-rw-r--r-- 1 ec2-user ec2-user 18 Dec 24 16:58 /backup/jenkins/test_xxx
ec2-user@ip-172-31-21-202 jenkins]$
```

✓ Extract the backup file using xvf and backup file name

Validation:

✓ Backup was restored in new Jenkins after refreshing the tab (/restart) in url after the url

The screenshot shows the Jenkins web interface. At the top, there's a navigation bar with 'Jenkins / Manage Jenkins / Nodes'. Below this, the 'Nodes' section is active, displaying a table with columns: S, Name, Architecture, Clock Difference, Free Disk Space, Free Swap Space, Free Temp Space, and Response Time. The table has one entry for the 'Built-In Node' with a status icon (a red square with a white 'x'). The 'Free Temp Space' column shows a red warning icon and '476.18 MiB'. Below the table, there's a 'Data obtained' row showing '55 sec' for several metrics.

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	Built-In Node	Linux (amd64)	In sync	5.06 GiB	0 B	476.18 MiB	0ms
Data obtained		55 sec	55 sec	55 sec	55 sec	55 sec	55 sec

ERROR SHEET:

- While creating node I have got error related to storage
- Jenkins node was failing because /tmp did not have enough space
- /tmp is RAM-based (tmpfs) on EC2
- Size is usually ~400–500 MB
- Jenkins requires ≥ 1 GB temp space for:
- Node connection

The screenshot shows the Jenkins 'Built-In Node' configuration page. At the top, there's a 'Bring this node back online' button and an 'Update offline reason' button. Below this, a red warning icon is followed by the text: 'Disk space is below threshold of 1.00 GiB. Only 473.29 MiB out of 477.99 MiB left on /tmp.' Below the warning, there's a 'Monitoring Data' dropdown menu.

Built-In Node Bring this node back online Update offline reason

This is the Jenkins controller's built-in node. Builds running on this node will execute on the same system and as the same user as the Jenkins controller. This is appropriate e.g. for special jobs performing backups, but in general you should run builds on agents. [Learn more about distributed builds.](#)

Disk space is below threshold of 1.00 GiB. Only 473.29 MiB out of 477.99 MiB left on /tmp.

Monitoring Data

ROOT CAUSE (important to understand):

- /tmp is mounted as tmpfs (RAM-based filesystem)
Its size is ~1 GB by design
Jenkins requires minimum 1 GB free disk
My /tmp has 950 MB, which is below Jenkins threshold
- This has NOTHING to do with EBS disk size
Even with 20 GB disk, /tmp remains ~1 GB because it's tmpfs

- That's why Jenkins still says:
- Disk space is below threshold of 1.00 GiB on /tmp

- **Step 1 (BEST & RECOMMENDED): Tell Jenkins NOT to use /tmp**

- Jenkins uses /tmp by default for temp files.
We will redirect Jenkins temp directory to disk.

- ◆ **Step 1: Create temp directory on disk**

- `sudo mkdir -p /var/lib/jenkins/tmp`
- `sudo chown jenkins:jenkins /var/lib/jenkins/tmp`

- - ◆ **Step 2: Add JVM temp override (systemd)**

- Edit override file:
- `sudo vi /etc/systemd/system/jenkins.service.d/override.conf`
- Make it **exactly like this**:
- [Service]
- `Environment="JENKINS_PORT=8081"`
- `Environment="JAVA_OPTS=-Djava.io.tmpdir=/var/lib/jenkins/tmp"`
- Save & exit.

- - ◆ **Step 3: Reload & restart Jenkins**

- `sudo systemctl daemon-reload`
- `sudo systemctl restart jenkins`

- - ◆ **Step 4: Bring node online**

- Jenkins UI → Built-In Node → Bring this node back online



Jenkins / Manage Jenkins / Nodes / New node

New node

Node name
Slave_1

Type
☒ Permanent Agent
Adds a plain, permanent agent to Jenkins. This is called "permanent" because Jenkins doesn't provide higher level of integration with these agents, such as dynamic provisioning. Select this type if no other agent types apply — for example such as when you are adding a physical computer, virtual machines managed outside Jenkins, etc.

Create